

AI Business Copilot (Agentic + RAG)

What it does:

- Takes business query
- Uses multiple agents:
 - Research Agent
 - Analysis Agent
 - Summary Agent
- Uses RAG for context
- Gives structured output

Stack:

- LangGraph / CrewAI
- Vector DB
- FastAPI
- Optional UI

Side note:

- Shows Agentic AI
- Shows RAG
- Shows architecture thinking
- Shows business understanding

Another Side Note

Structure:

- Problem
- Solution
- Architecture
- Impact

Problem:

Teams struggled to find information across documents.

Solution:

Built a RAG-based AI assistant using LangChain.

Architecture:

User → Embedding → Vector DB → LLM

Impact:

Reduced search time by 60%

WHAT THIS SYSTEM WILL DO (Clear Vision)

User input:

“Analyze why sales dropped last quarter and suggest actions”

System output:

- Root cause analysis
- Data-backed reasoning (via RAG)
- Actionable recommendations

System Architecture :

User Query

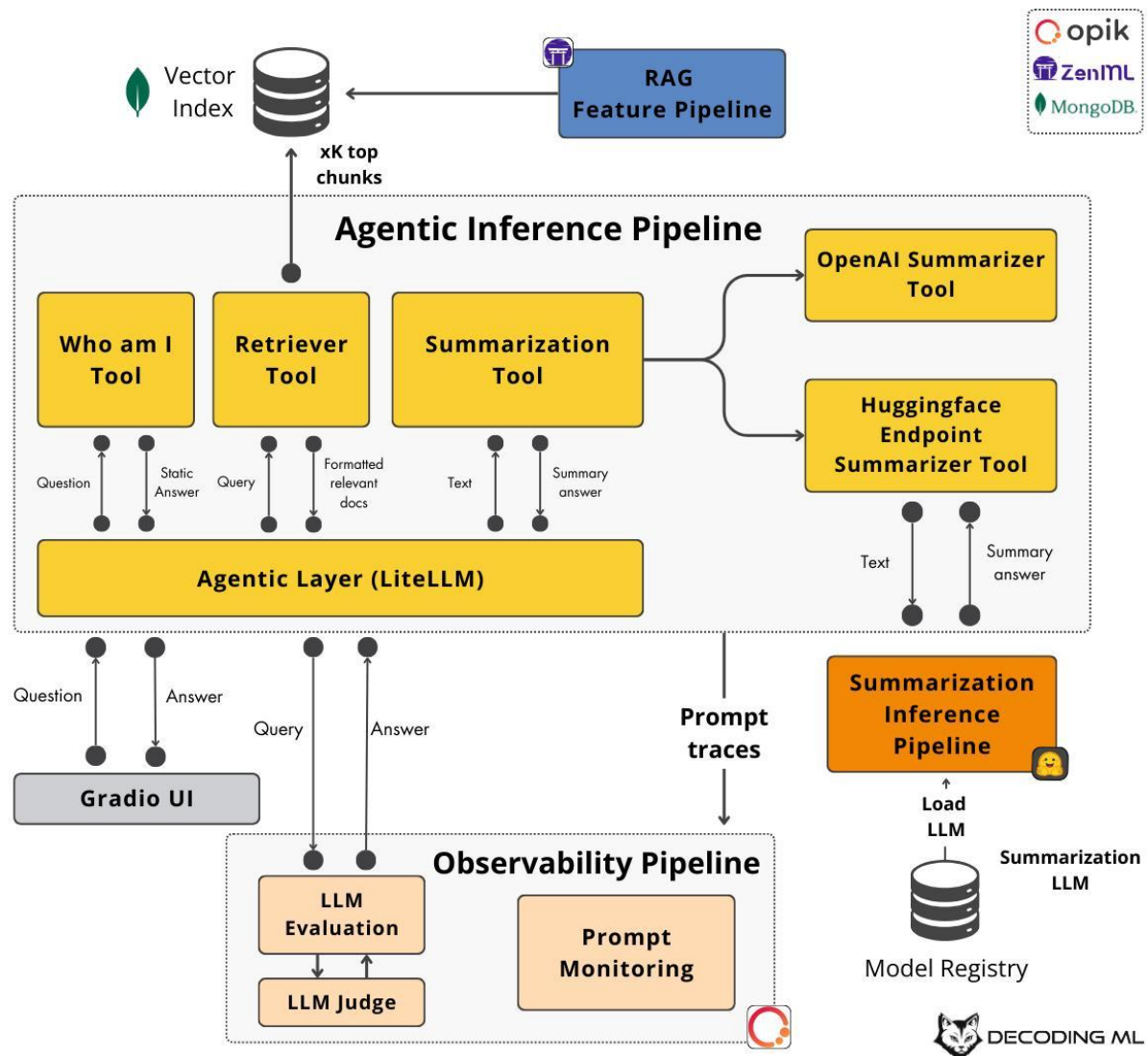
Router Agent decides flow

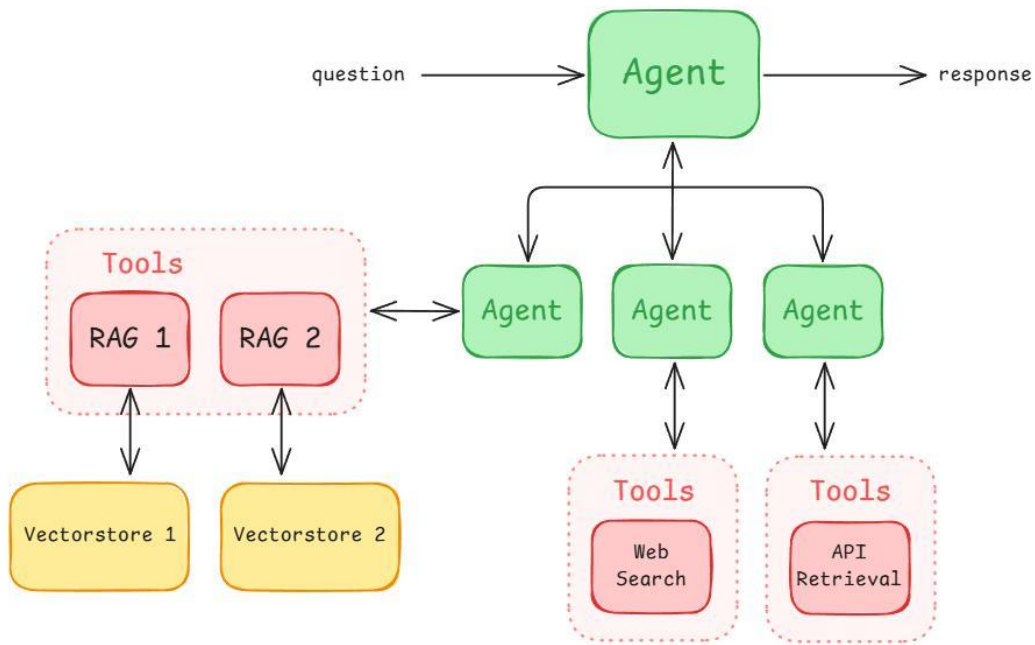
RAG retrieves context

Agents collaborate:

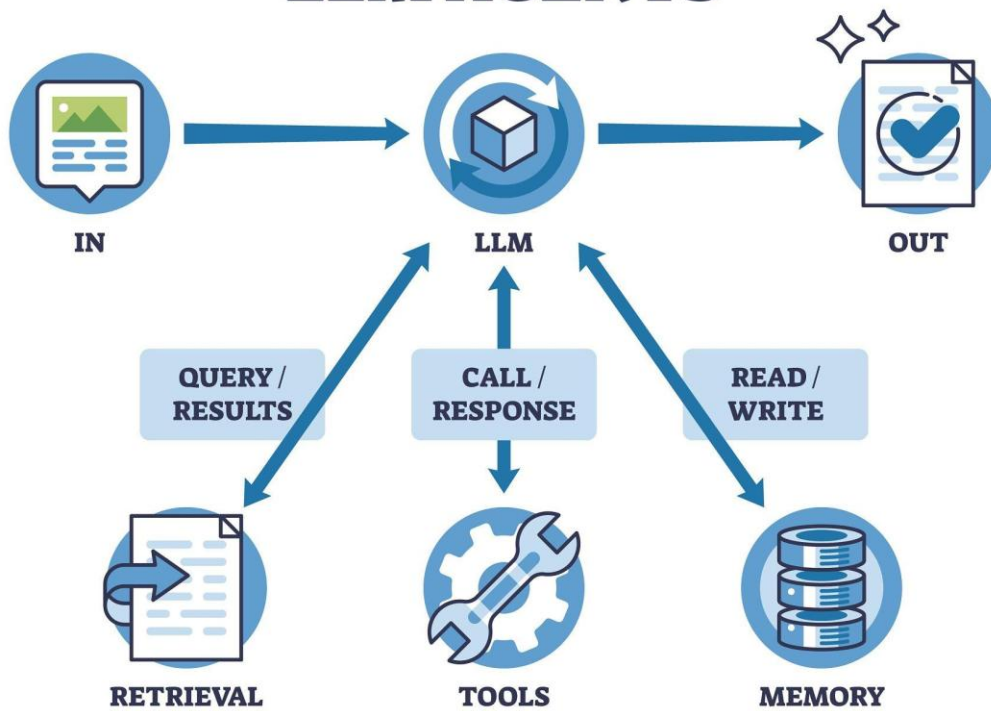
- Research Agent
- Analyst Agent
- Strategy Agent

Final Response Generator



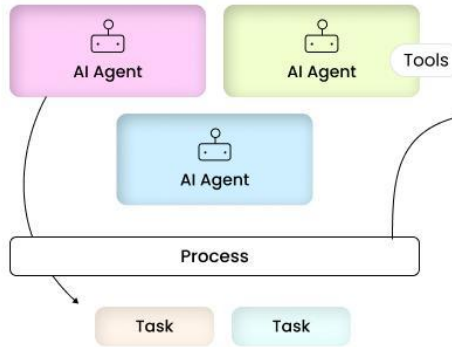


LLM AGENTS



Crew

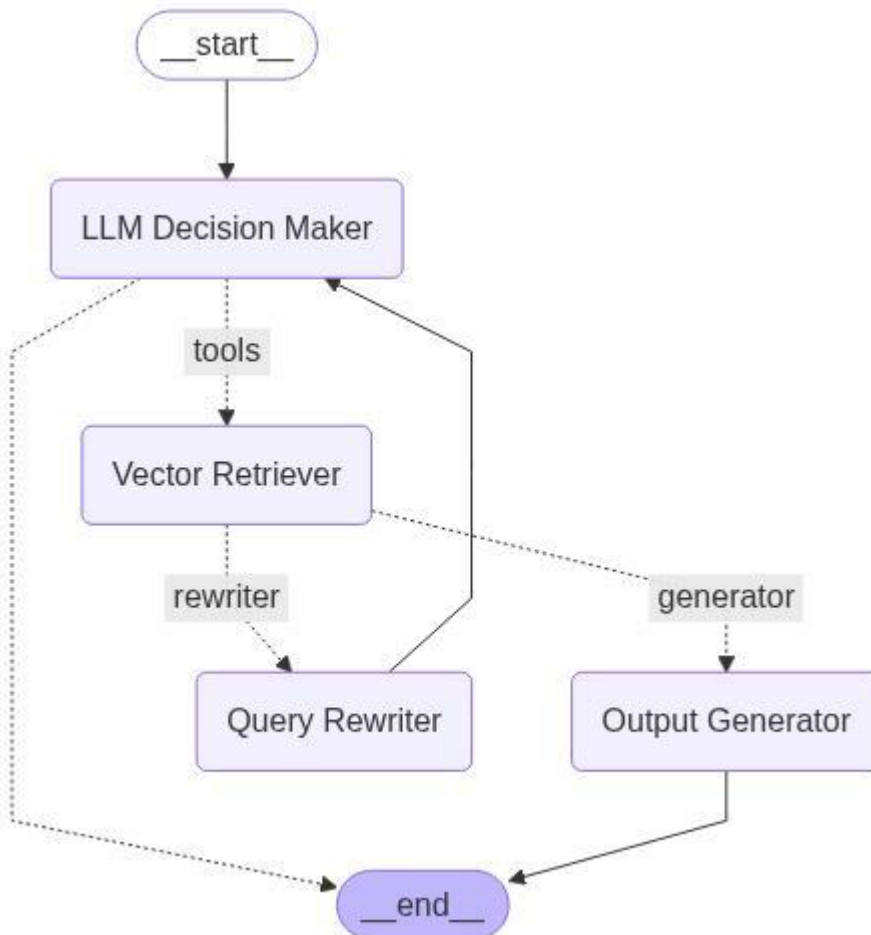
Agents have the inherent ability to reach out to one another to delegate work or ask questions.

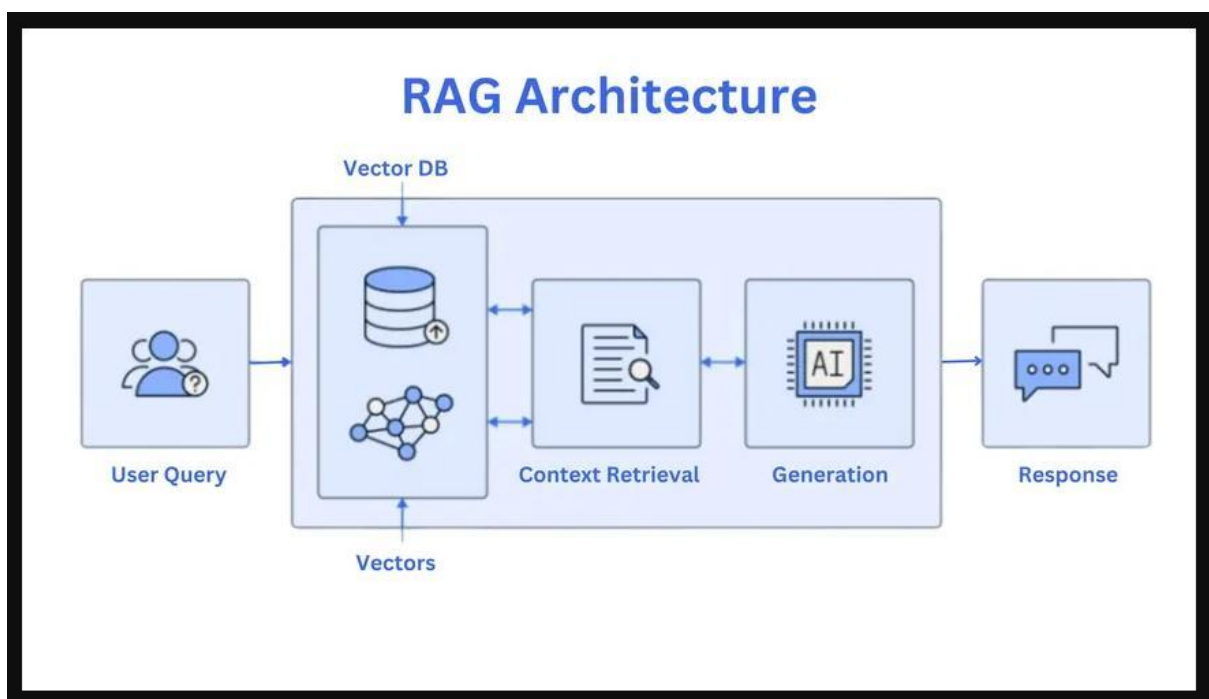
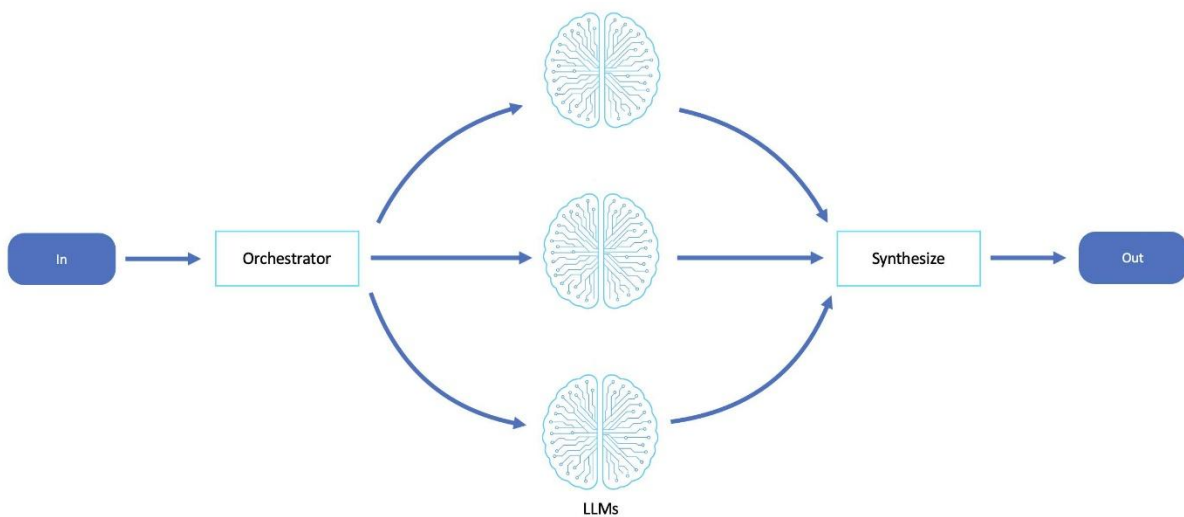


Processes define how agents will work together. How tasks will be assigned. Interaction with each other. How they will perform work.

Tasks can override agent tools with specific ones that should be used and also have a specific agent tackle them.

Outcome





Tech Stack

Core:

- LangGraph (preferred) OR CrewAI
- LangChain (RAG pipeline)

Backend:

- FastAPI

LLM:

- OpenAI / Azure OpenAI

Vector DB:

- FAISS / Pinecone

Optional UI:

- Streamlit / React

AGENT DESIGN

1. Router Agent

Role:

- Understand user intent
- Decide which agents to activate

2. Research Agent

Role:

- Fetch relevant data using RAG
- Query vector DB

3. Analyst Agent

Role:

- Analyze retrieved data
- Identify patterns, issues

4. Strategy Agent

Role:

- Generate recommendations

- Business-focused output

5. Response Agent

Role:

- Format final answer cleanly

5. PROJECT STRUCTURE (CLEAN + PROFESSIONAL)

```
ai-business-copilot/  
|  
├─ app/  
|   ├── main.py (FastAPI entry)  
|   ├── routes/  
|   ├── services/  
|  
├─ agents/  
|   ├── router_agent.py  
|   ├── research_agent.py  
|   ├── analyst_agent.py  
|   ├── strategy_agent.py  
|  
├─ rag/  
|   ├── embeddings.py  
|   ├── vector_store.py  
|   ├── retriever.py  
|  
├─ workflows/  
|   └─ langgraph_flow.py
```



```
|  
|  
├─ utils/  
|  
├─ tests/  
|  
└─ README.md
```

6. CORE IMPLEMENTATION (SIMPLIFIED FLOW)

```
# pseudo-code  
  
def router(state):  
    return "research"  
  
def research(state):  
    docs = retrieve_from_vector_db(state["query"])  
    return {"context": docs}  
  
def analyze(state):  
    return {"analysis": llm_analyze(state["context"])}  
  
def strategy(state):  
    return {"recommendations": llm_strategy(state["analysis"])}  
  
def respond(state):  
    return format_output(state)  
  
graph = StateGraph()  
  
graph.add_node("router", router)  
graph.add_node("research", research)  
graph.add_node("analyze", analyze)  
graph.add_node("strategy", strategy)  
graph.add_node("respond", respond)  
  
graph.set_entry_point("router")
```

7. RAG SETUP (MINIMUM VIABLE)

Use:

- PDFs (business reports)
- Sample company docs

Flow:

- Chunk → Embed → Store → Retrieve

8. OUTPUT FORMAT

Problem Analysis:

- Sales declined due to reduced demand in segment X

Key Insights:

- 30% drop in region Y
- Competitor pricing lower

Recommendations:

1. Adjust pricing strategy
2. Focus on high-performing regions
3. Improve marketing campaigns

Reporting Structure:

- Problem
- Solution

- Architecture
 - Tech stack
 - Demo
-

5 Days Plan

Day 1:

- Setup repo + RAG

Day 2:

- Build agents

Day 3:

- LangGraph workflow

Day 4:

- FastAPI + UI

Day 5:

- README + Demo video